Classification bayésienne supervisée de pixels : analyse des espaces colorimétriques RGB, HSV et Y′CbCr

1st Jair Anderson Vasquez Torres
Ingénieur Degree Programme STIC
ENSTA Paris
Paris, France
jair-anderson.vasquez@ensta.fr

2nd Santiago Florido Gomez
Ingénieur Degree Programme STIC
ENSTA Paris
Paris, France
santiago.florido@ensta-paris.fr

***Abstract*—Ce travail présente un cadre de classification bayésienne supervisée appliqué à la segmentation de pixels en imagerie couleur. L'article formalise les critères de décision Maximum A Posteriori (MAP) et Maximum Vraisemblance (ML), puis étudie le rôle du choix de l'espace d'observation dans la séparabilité statistique des classes. Trois représentations colorimétriques sont analysées : RGB, HSV et Y′CbCr. Nous décrivons leurs transformations, leurs propriétés pertinentes pour l'inférence probabiliste, et l'extraction opérationnelle de leurs composantes au moyen d'une implémentation Python/OpenCV. Les visualisations et canaux exportés (`R,G,B,H,S,V,Y′,Cb,Cr`) fournissent une base reproductible pour la construction de modèles bayésiens de classification de pixels et la comparaison de performances entre espaces colorimétriques.**

***Index Terms*—Classification bayésienne, Maximum A Posteriori, Maximum Vraisemblance, segmentation de pixels, RGB, HSV, Y′CbCr, OpenCV.**

I. Convolutional neural networks et CIFAR-10

Dans le contexte de la classification d'images, les réseaux de neurones convolutionnels se sont consolidés comme l'une des architectures les plus adaptées à l'analyse des données visuelles, car ils permettent l'extraction automatique de caractéristiques spatiales hiérarchiques à partir des données d'entrée. Un réseau neuronal implémenté peut être modélisé comme une application composée

$$f_\theta : \mathbb{R}^{H\times W\times C} \to \mathbb{R}^K, \tag{1}$$

où H et W sont la hauteur et la largeur de l'image, C est le nombre de canaux, K est le nombre de classes et θ désigne l'ensemble des paramètres entraînables.

Cette architecture remplace la connexion dense d'un réseau totalement connecté par une structure à champs réceptifs locaux, partage de poids et extraction hiérarchique de caractéristiques. Une CNN est organisée en plusieurs étapes, et chaque étape contient essentiellement une couche de filtres, une non-linéarité et une opération de réduction ou de *pooling* sur des cartes de caractéristiques [2].

Soit $a^{(0)} = x$ l'entrée. Dans une couche convolutionnelle l, si l'entrée possède C_{l-1} cartes et la sortie possède C_l cartes, l'opération peut s'écrire

$$z^{(l)}_{i,j,m} = b^{(l)}_m + \sum_{c=1}^{C_{l-1}} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K^{(l)}_{u,v,c,m}\, a^{(l-1)}_{i+u,\, j+v,\, c}, \tag{2}$$

et, après la convolution, une non-linéarité ponctuelle est appliquée :

$$a^{(l)}_{i,j,m} = \phi\Big(z^{(l)}_{i,j,m}\Big). \tag{3}$$

La dimension spatiale de la sortie d'une convolution avec pas (*stride*) s et remplissage (*padding*) p est donnée par

$$H_l = \left\lfloor \frac{H_{l-1} + 2p_h - k_h}{s_h} \right\rfloor + 1, \qquad W_l = \left\lfloor \frac{W_{l-1} + 2p_w - k_w}{s_w} \right\rfloor + 1. \tag{4}$$

Cela permet de contrôler la résolution spatiale à mesure que le signal traverse le réseau.

Si un réseau possède plusieurs couches, le champ réceptif effectif d'un neurone dans les couches profondes croît avec la profondeur. Une manière utile de le modéliser est donnée par la récurrence

$$R_1 = k_1, \qquad R_l = R_{l-1} + (k_l - 1)\prod_{t=1}^{l-1} s_t, \tag{5}$$

où R_l est la taille du champ réceptif effectif dans la couche l.

L'un des principaux avantages quantitatifs de la convolution est son efficacité paramétrique. Si une couche convolutionnelle possède C_{in} canaux d'entrée, C_{out} canaux de sortie et des filtres de taille $k_h \times k_w$, alors le nombre de paramètres est

$$N^{\text{conv}}_{\text{params}} = (k_h k_w C_{\text{in}} + 1)\, C_{\text{out}}. \tag{6}$$

En revanche, une couche dense reliant une entrée de taille n_{in} à une sortie de taille n_{out} possède

$$N^{\text{fc}}_{\text{params}} = (n_{\text{in}} + 1)\, n_{\text{out}}. \tag{7}$$

Cette différence constitue un élément significatif dans la classification automatique d'images, puisqu'une image d'entrée peut contenir des milliers de variables, tandis que

la convolution exploite leurs caractéristiques structurelles en évitant une croissance explosive du nombre de poids.

Dans l'étape finale de classification, la représentation obtenue par les couches convolutionnelles est aplatie ou agrégée globalement afin de produire un vecteur $h \in \mathbb{R}^d$. Ensuite, une couche affine calcule

$$z = Wh + b, \qquad z \in \mathbb{R}^K, \tag{8}$$

et la probabilité estimée pour la classe k est obtenue au moyen de la fonction *softmax* :

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}. \tag{9}$$

[2]

Il est proposé d'utiliser la base de données CIFAR-10, largement employée dans l'évaluation des algorithmes de classification d'images, qui contient $60\,000$ images RGB de 32×32 pixels réparties en 10 classes, avec $6\,000$ images par classe. La partition standard comprend $50\,000$ images pour l'entraînement et $10\,000$ pour le test ; de plus, l'ensemble de test contient exactement $1\,000$ images par classe et l'ensemble d'entraînement en contient $5\,000$ par classe. Les classes sont *airplane*, *automobile*, *bird*, *cat*, *deer*, *dog*, *frog*, *horse*, *ship* et *truck* [3], ce qui représente un scénario adéquat pour l'analyse des éléments fondamentaux de l'apprentissage profond avec des exigences computationnelles modérées.

Comme une partie de l'objectif de recherche consiste également à étudier de manière contrôlée l'influence de la préparation des données, de la définition de l'architecture, de la sélection des hyperparamètres et de la définition de la dynamique d'apprentissage sur la performance idéale du modèle implémenté, ce cadre expérimental se révèle particulièrement pertinent.

II. Structuration des données

A. *training, validation, test*

La division de la base de données constitue un élément méthodologique propre à l'apprentissage supervisé, de manière à isoler l'information destinée à l'ajustement des paramètres, à la sélection du modèle et à l'évaluation du modèle entraîné. L'ensemble d'entraînement est utilisé pour l'estimation des paramètres du réseau, l'ensemble de validation a pour objectif de mesurer la généralisation du modèle sans intervenir directement dans les actualisations des paramètres, et, enfin, l'ensemble de test sert à obtenir une estimation indépendante et non biaisée de la performance du modèle sur des données qu'il n'a pas vues.

L'objectif de cette séparation est d'éviter la fuite d'information, qui peut se produire lorsque les données d'évaluation du modèle ont été utilisées dans l'estimation des paramètres et dans la définition de celui-ci, afin de pouvoir disposer de véritables mesures de la capacité de généralisation du modèle entraîné.

B. *fonction standardize*

La fonction `standardize` peut être comprise comme une standardisation spatiale réalisée image par image et canal par canal. Pour un ensemble d'images représenté par un tenseur $x \in \mathbb{R}^{N \times H \times W \times C}$, la fonction calcule, pour chaque image n et pour chaque canal c, la moyenne $\mu_{n,c}$ et l'écart-type $\sigma_{n,c}$ sur les dimensions spatiales (H, W), puis transforme chaque pixel selon l'expression (10) :

$$x'_{n,i,j,c} = \frac{x_{n,i,j,c} - \mu_{n,c}}{\sigma_{n,c}}. \tag{10}$$

L'implication de cette implémentation est que chaque canal de chaque image est recentré autour de zéro et redimensionné de sorte que la dispersion soit aussi proche que possible d'une dispersion unitaire. Cet effet d'homogénéisation de l'entrée, en termes de son échelle numérique, facilite le processus d'optimisation et peut être utile pour accroître la stabilité de l'entraînement, car, en réduisant les différences globales d'intensité et de contraste entre les images, la standardisation diminue la variabilité non informative mais présente dans les données d'entrée, de manière à favoriser l'apprentissage par le réseau des motifs structurels les plus pertinents.

III. Architecture du réseau

Dans cette architecture, on définit un réseau de neurones convolutionnel séquentiel composé d'une étape d'extraction de caractéristiques locales au moyen de la convolution, suivie de réductions dimensionnelles par *max pooling*. La couche d'entrée est définie pour un tenseur d'entrée $32 \times 32 \times 3$, où 32 est la hauteur, 32 la largeur et 3 le nombre de canaux de couleur (R, G, B). Cette couche n'apprend pas de paramètres ; elle fixe uniquement la forme attendue des données d'entrée du modèle [6].

Ensuite, une première couche convolutionnelle est définie pour la première extraction des caractéristiques. La couche apprend 8 filtres distincts, ce qui implique qu'à sa sortie elle aura 8 cartes de caractéristiques. Une dimension de noyau $(3, 3)$ est définie, et l'argument `padding="same"` est incorporé dans la définition, ce qui permet de préserver la résolution spatiale de l'image, de sorte que la sortie conserve une hauteur et une largeur égales à celles de l'entrée. Par conséquent, la sortie de cette couche a la taille

$$32 \times 32 \times 8. \tag{11}$$

Une couche de *dropout* est ajoutée, laquelle remplit la fonction de mécanisme de régularisation durant l'entraînement en annulant aléatoirement une fraction des activations des entrées. Néanmoins, puisque la valeur du *dropout* est fixée par défaut à zéro, la couche ne modifie ni l'information ni la dimension du tenseur. Sa sortie demeure

$$32 \times 32 \times 8. \tag{12}$$

On ajoute une couche de *max pooling* qui a un *pool size* $(2, 2)$, ce qui implique qu'elle réduit la résolution spatiale en

prenant la valeur maximale dans chaque fenêtre locale 2×2. De cette manière, la sortie de cette couche est

$$16 \times 16 \times 8. \tag{13}$$

On ajoute ensuite une couche qui convertit le vecteur tridimensionnel en un vecteur unidimensionnel de dimension

$$16 \cdot 16 \cdot 8 = 2048, \tag{14}$$

puis une transformation *fully connected* est implémentée au moyen d'une couche dense vers un espace latent de 64 neurones :

$$\mathbf{h} = \mathrm{ReLU}(W\mathbf{x} + \mathbf{b}). \tag{15}$$

De sorte que la sortie de cette couche est de taille 64, on ajoute une deuxième couche de *dropout* dans laquelle le *rate* est fixé à 0, pour finalement inclure une couche dense supplémentaire vers un espace latent de dimension 10. Dans cette occasion, l'activation *softmax* transforme les scores en une distribution de probabilité :

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{10} e^{z_j}}, \qquad k = 1, \ldots, 10. \tag{16}$$

Par conséquent, chaque composante de la sortie appartient à l'intervalle $[0, 1]$ et la somme de toutes est égale à 1. La classe prédite est celle associée à l'indice ayant la probabilité la plus élevée :

$$\hat{c} = \arg\max_k \hat{y}_k. \tag{17}$$

Enfin, il y a une dernière couche de *dropout* avec un taux égal à 0.

Les paramètres entraînables sont ceux que le modèle met à jour durant l'entraînement au moyen de la rétropropagation et de l'optimisation, lesquels, dans le cas de la structure proposée, sont incorporés dans les couches convolutionnelles et denses, puisque des couches telles que `Input`, `Dropout`, `MaxPooling2D` et `Flatten` n'incorporent pas de paramètres entraînables dans leur définition. Dans le cas de la couche `Conv2D`,

$$N_{\text{params}} = (k_h \cdot k_w \cdot C_{\text{in}} + 1) \cdot C_{\text{out}}, \tag{18}$$

où k_h et k_w sont les dimensions du filtre, C_{in} le nombre de canaux d'entrée, et C_{out} le nombre de filtres de sortie. Ici,

$$k_h = 3, \qquad k_w = 3, \qquad C_{\text{in}} = 3, \qquad C_{\text{out}} = 8. \tag{19}$$

Alors,

$$N_{\text{params}} = (3 \cdot 3 \cdot 3 + 1) \cdot 8 = (27 + 1) \cdot 8 = 224. \tag{20}$$

C'est-à-dire que cette couche possède 224 paramètres entraînables, y compris le biais [4].

Pour sa part, dans la couche dense avec biais,

$$N_{\text{params}} = (n_{\text{in}} + 1) \cdot n_{\text{out}}, \tag{21}$$

où n_{in} est le nombre d'entrées et n_{out} le nombre de neurones de sortie. Dans le cas de la structure proposée, on a deux couches denses :

$$N_{\text{params}} = (2048 + 1) \cdot 64 = 2049 \cdot 64 = 131136, \tag{22}$$

et

$$N_{\text{params}} = (64 + 1) \cdot 10 = 650. \tag{23}$$

Ainsi, le nombre total de paramètres entraînables est

$$224 + 131136 + 650 = 132010. \tag{24}$$

IV. Apprentissage

A. Définitions : epoch, step, batch

Conformément au cours, l'apprentissage supervisé du CNN de la Section III consiste à ajuster un ensemble de poids entraînables

$$W = \{w_{ij}\}, \tag{25}$$

sur l'ensemble d'apprentissage $A = \{(X_i, V_i)\}_{i=1}^{N}$, de manière à minimiser la loss empirique

$$L_A(W) = \frac{1}{N} \sum_{i=1}^{N} L(\mathrm{ForwardProp}(W, X_i), V_i)\,, \qquad \widehat{W} = \arg\min_W L_A(W \tag{26}$$

Lorsque A est partitionné en lots $A_b \subset A$, $|A_b| = B$, la loss associée à un lot est définie par

$$L_b(W) = \frac{1}{|A_b|} \sum_{(X_i, V_i) \in A_b} L(\mathrm{ForwardProp}(W, X_i), V_i)\,. \tag{27}$$

Cette normalisation par $|A_b|$ est retenue dans toute la section, car elle permet de comparer des batch sizes différents sans modifier artificiellement l'échelle numérique de la loss ; elle correspond en outre à la convention de réduction `sum_over_batch_size` utilisée dans l'implémentation Keras.

Une itération d'optimisation sur un lot s'écrit alors, pour chaque connexion (i, j),

$$w_{ij}^{(t)} = w_{ij}^{(t-1)} - \epsilon \frac{\partial L_b}{\partial w_{ij}}, \tag{28}$$

où $\epsilon > 0$ désigne le pas d'apprentissage. Dans ce cadre, un *batch* est un sous-ensemble A_b de cardinalité B, un *step* est une mise à jour de la forme (28), et une *epoch* correspond à un parcours complet de la base d'apprentissage. Si N exemples sont traités avec des lots de taille B, le nombre de steps par epoch vaut

$$S(B) = \left\lceil \frac{N}{B} \right\rceil. \tag{29}$$

Le cours distingue alors trois régimes : $B = 1$ correspond au gradient stochastique, $B = N$ au gradient par lots complets, et $1 < B < N$ à l'apprentissage par mini-batches, qui recherche un compromis entre bruit du gradient, coût calculatoire et vitesse de convergence [7], [8].

B. Influence de la taille du batch

a) Protocole expérimental.: La présente section ne redéfinit ni les données ni l'architecture : le découpage des sous-ensembles et la fonction `standardize` sont ceux de la Section II, et le CNN entraîné est celui de la Section III. Seuls les paramètres d'apprentissage sont modifiés pour l'étude des batch sizes et des optimiseurs.

TABLE I
PROTOCOLE EXPÉRIMENTAL DE LA SECTION 4.

Références amont	Section II pour les données et la standardisation ; Section III pour l'architecture
Sous-ensembles utilisés	5000 images d'entraînement, 1000 images de validation ; l'ensemble de test n'intervient pas dans cette section
Graine de partition	42
Graines d'initialisation	42, 314
Comparaison des batch sizes	$B \in \{8, 16, 32, 64, 128\}$, 8 epochs fixes, SGD avec $\epsilon = 10^{-2}$
Comparaison des optimiseurs	$B = 32$, 8 epochs fixes, mêmes poids initiaux pour une même graine
Hyperparamètres optimiseurs	SGD : $\epsilon = 10^{-2}$; SGD+Momentum : $\epsilon = 10^{-2}$, $\beta = 0.9$; Adam : $\epsilon = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-7}$
Conventions temporelles	temps/step : moyenne des temps mur mesurés sur les lots d'entraînement ; temps/epoch : temps mur d'une epoch incluant la validation

La comparaison de la taille du batch est conduite à nombre d'epochs fixé, et non à nombre de mises à jour fixé. Ce point est essentiel pour l'interprétation : sur 8 epochs, le nombre total d'updates vaut $8S(B)$, soit 5000, 2504, 1256, 632 et 320 pour $B = 8$, 16, 32, 64 et 128. Les différences observées entre batch sizes combinent donc deux effets : la variation de bruit du gradient et la variation du nombre total de mises à jour.

Dans le tableau II, le symbole $m \pm \sigma$ désigne la moyenne empirique m et l'écart-type empirique σ ; pour le temps par step, l'agrégation porte sur l'ensemble des steps mesurés au cours des deux graines et des huit epochs, tandis que pour le temps par epoch elle porte sur l'ensemble des epochs observées. Les courbes de la figure 3 sont les moyennes calculées sur les deux graines.

TABLE II
INFLUENCE DE LA TAILLE DU BATCH SUR LES COÛTS TEMPORELS MOYENS.

Batch size B	Steps/epoch $S(B)$	Temps moyen par step (s)	Temps moyen par epoch (s)
8	625	0.0052 ± 0.0107	3.882 ± 0.460
16	313	0.0079 ± 0.0160	2.959 ± 0.391
32	157	0.0117 ± 0.0234	2.234 ± 0.410
64	79	0.0182 ± 0.0287	1.745 ± 0.371
128	40	0.0297 ± 0.0383	1.457 ± 0.340

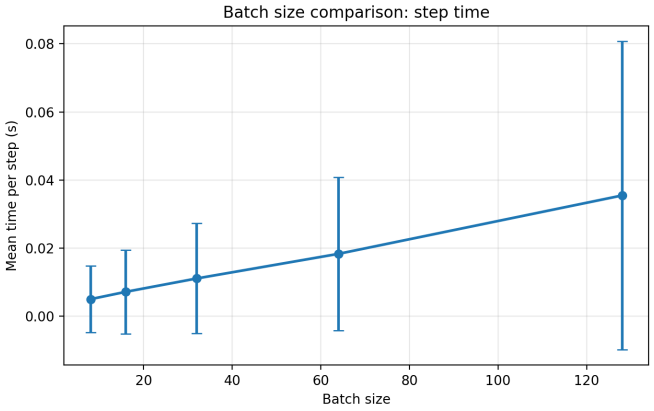

Fig. 1. Temps moyen par step en fonction de la taille du batch.

Les figures 1 et 2 confirment la relation attendue par (29). Le temps moyen par step croît avec B, de $0.0052\,\mathrm{s}$ à $0.0297\,\mathrm{s}$,

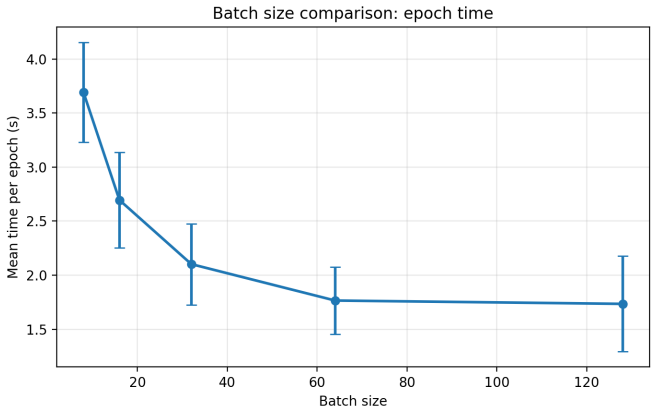

Fig. 2. Temps moyen par epoch en fonction de la taille du batch.

car chaque mise à jour traite davantage d'exemples. En sens inverse, le temps moyen par epoch décroît de $3.882\,\mathrm{s}$ à $1.457\,\mathrm{s}$, puisque le nombre de steps diminue fortement. Cette décroissance reste sous-linéaire, ce qui est cohérent avec le cours : l'augmentation de la taille du lot réduit le nombre d'updates mais n'annule pas le surcoût de calcul de chaque update [7], [8].

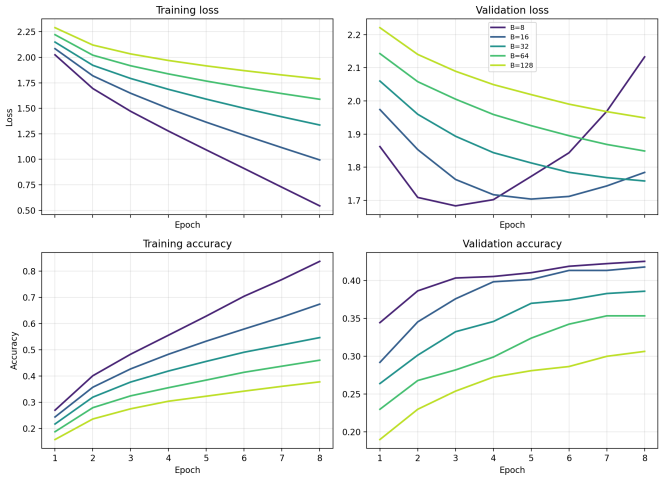

Fig. 3. Courbes moyennes d'entraînement et de validation pour différentes tailles de batch.

L'interprétation des courbes doit rester liée au budget fixé en epochs. Dans ce cadre, les plus petits batches atteignent les meilleures accuracies de validation finales : 42.55% pour $B = 8$ et 41.80% pour $B = 16$, contre 38.60%, 35.35% et 30.65% pour $B = 32$, 64 et 128. Une partie de cet avantage provient du fait que les petits batches réalisent un nombre nettement plus grand de mises à jour sur le même budget de 8 epochs. En revanche, $B = 8$ produit aussi la dynamique de validation la plus bruitée : la val. loss atteint un minimum moyen de 1.6833 à l'epoch 3, puis remonte jusqu'à 2.1329 à l'epoch 8. Sous le protocole considéré, $B = 16$ apparaît ainsi comme la valeur la plus équilibrée parmi celles testées,

en conservant une accuracy finale proche de $B = 8$ avec des courbes de validation moins irrégulières et un temps par epoch plus faible.

C. Comparaison des méthodes d'optimisation

Le SGD de base correspond directement à l'actualisation scalaire (28), appliquée à chaque connexion (i, j). Le cours présente ensuite le SGD avec Momentum sous la forme d'une accumulation des gradients successifs ; pour chaque poids w_{ij},

$$G_t = \beta G_{t-1} + (1-\beta)\frac{\partial L_b}{\partial w_{ij}}, \qquad w_{ij}^{(t)} = w_{ij}^{(t-1)} - \epsilon G_t. \quad (30)$$

Le terme G_t lisse ainsi les variations de direction du gradient d'un batch au suivant [7], [8].

Pour Adam, la présente section reprend explicitement l'écriture simplifiée du cours, sans correction de biais :

$$G_t = \beta_1 G_{t-1} + (1-\beta_1)\frac{\partial L_b}{\partial w_{ij}}, \quad (31)$$

$$K_t = \beta_2 K_{t-1} + (1-\beta_2)\left(\frac{\partial L_b}{\partial w_{ij}}\right)^2, \quad (32)$$

$$w_{ij}^{(t)} = w_{ij}^{(t-1)} - \epsilon\,\frac{G_t}{\sqrt{K_t}+\eta}. \quad (33)$$

Dans cette expression, ϵ désigne le pas d'apprentissage, tandis que $\eta > 0$ est le petit terme introduit au dénominateur pour èviter une division par zéro. Cette méthode adapte donc, pour chaque poids, l'amplitude de la mise à jour tout en lissant les fluctuations temporelles du gradient.

La comparaison des optimiseurs est menée à batch size fixé $B = 32$ et à nombre d'epochs fixé (8). Le tableau III regroupe les statistiques agrégées sur les deux graines ; pour le temps par epoch, la notation $m \pm \sigma$ correspond à la moyenne et à l'écart-type empiriques sur l'ensemble des epochs mesurées, tandis que l'accuracy finale de validation et le minimum de val. loss sont moyennés sur les deux graines.

TABLE III
COMPARAISON DES OPTIMISEURS À BATCH SIZE FIXÉ $B = 32$. LE SYMBOLE $\pm$ DÉSIGNE L'ÉCART-TYPE EMPIRIQUE DU TEMPS PAR EPOCH.

Optimiseur	Temps moyen par epoch (s)	Accuracy finale de validation (%)	Minimum de la val. loss
SGD	2.133 $\pm$ 0.334	38.60	1.7584
SGD+Momentum	2.240 $\pm$ 0.493	40.20	1.7079
Adam	2.417 $\pm$ 0.755	43.45	1.6083

Sous ce protocole, le SGD fournit la décroissance de validation la plus régulière : la val. loss baisse presque monotoniquement jusqu'à 1.7584, pour une accuracy finale de 38.60%. Le Momentum accélère la descente initiale, avec un minimum moyen de val. loss à 1.7079 dès l'epoch 2, mais la remontée ultérieure jusqu'à 2.6743 à l'epoch 8 indique une dégradation plus marquée en fin d'entraînement. Cette observation est cohérente avec l'interprétation du cours : le terme G_t stabilise les directions du gradient, mais peut aussi prolonger des mises à jour devenues trop agressives lorsque ϵ reste fixé.

Parmi les trois optimiseurs testés, Adam obtient les meilleures métriques de validation dans le cadre présent :

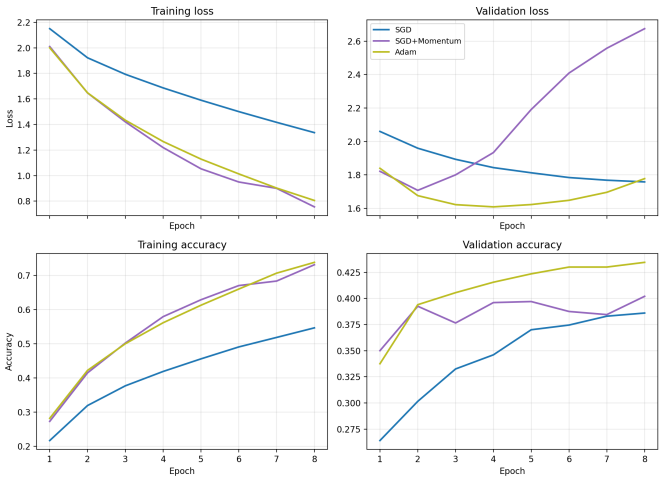

Fig. 4. Courbes moyennes d'entraînement et de validation pour SGD, SGD+Momentum et Adam.

accuracy finale moyenne de 43.45% et minimum moyen de val. loss de 1.6083. Le surcoût temporel reste modéré ($2.417\,\mathrm{s}$ par epoch contre $2.133\,\mathrm{s}$ pour SGD), ce qui suggère que l'adaptation locale des mises à jour compense efficacement cette surcharge sur le budget calculatoire considéré.

D. Synthèse

Sous le protocole à nombre d'epochs fixé retenu ici, la taille du batch agit simultanément sur le bruit du gradient et sur le nombre total de mises à jour. Les petits batches favorisent donc la performance de validation, mais au prix de courbes plus bruitées et d'un temps par epoch plus élevé ; parmi les valeurs testées, $B = 16$ fournit le compromis le plus favorable entre coût temporel et qualité de validation, tandis que $B = 8$ donne l'accuracy finale la plus élevée.

a) Limites expérimentales.: La portée de ces conclusions reste limitée au protocole étudié : sous-ensemble réduit de CIFAR-10, CNN de taille modeste, apprentissage sur CPU et deux graines d'initialisation seulement. Dans ce cadre, Adam présente les meilleures métriques de validation parmi les optimiseurs comparés, avec un surcoût temporel modéré ; cette observation ne prétend toutefois pas valoir indépendamment de l'architecture, du budget d'updates ou du choix des hyperparamètres.

V. HYPERPARAMÈTRES

A. Définition et distinction avec les paramètres entraînables

Contrairement aux poids du réseau, détaillés à la Section III, les hyperparamètres ne sont pas appris par rétropropagation. Ils sont fixés avant ou autour de l'apprentissage et déterminent l'architecture, la fonction objective, le schéma d'optimisation, le protocole d'entraînement et la régularisation [8], [7]. Les poids $W = \{w_{ij}\}$ et les biais des couches convolutionnelles et denses sont, au contraire, ajustés pendant l'optimisation selon les èquations de la Section 4.

Les quantités de journalisation et de sauvegarde, telles que `verbose`, `filepath` ou les options de

`ModelCheckpoint`, sont donc exclues de l'inventaire. Les tailles de sous-ensembles, les graines et le prétraitement `standardize` influencent l'expérience, mais ils sont traités ici comme des conditions expérimentales externes et non comme des hyperparamètres du modèle au sens strict.

B. Inventaire analytique des hyperparamètres

L'inventaire est ètabli directement à partir des appels du notebook et de la pipeline reproductible, en relevant explicitement des expressions telles que `Conv2D(filters=8, kernel_size=(3,3), padding="same")`, `units` (Dense cachée), `compile(loss="categorical_crossentropy", optimizer=...)` et `fit(..., batch_size=..., epochs=..., shuffle=True)`. Pour gagner en précision, les hyperparamètres sont distingués entre hyperparamètres structurels imposés par la tâche, hyperparamètres ajustables de conception, hyperparamètres d'entraînement et hyperparamètres de régularisation.

Les tailles de sous-ensembles, les graines de partition ou d'initialisation et le prétraitement `standardize` ne figurent pas dans les Tables IV et V. Ces quantités modifient bien les conditions d'observation et la variance des résultats, mais elles ne constituent pas des hyperparamètres sintonisables du modèle lui-même.

C. Influence des principaux hyperparamètres sur l'apprentissage

a) Hyperparamètres d'optimisation.: Les hyperparamètres d'optimisation ont déjà fait l'objet d'une ètude empirique en Section 4 ; ils sont seulement replacés ici dans une typologie générale. Dans (28), le pas d'apprentissage ϵ multiplie directement le gradient. Un ϵ trop grand peut donc provoquer des oscillations ou faire franchir une zone de minimum ; un ϵ trop faible ralentit au contraire la convergence et augmente le coût calculatoire. Le choix de l'optimiseur modifie ensuite la manière dont cette actualisation est lissée ou repondérée : le Momentum introduit une inertie directionnelle, tandis qu'Adam adapte localement l'amplitude des mises à jour. La taille du batch, le nombre d'epochs et `shuffle` gouvernent enfin le bruit du gradient, le budget d'actualisations et les effets d'ordre entre mini-batches.

b) Hyperparamètres d'architecture.: Les hyperparamètres d'architecture déjà décrits en Section III deviennent ici des quantités de conception ajustables. Augmenter le nombre de filtres ou la dimension de la couche dense accroît la capacité de représentation du réseau, mais aussi le nombre de poids à estimer. Le noyau convolutionnel et le *pooling* contrôlent en outre le champ récepteur effectif des neurones profonds : via (5), ils déterminent la portion du contexte spatial initial que les dernières couches peuvent analyser pour prendre une décision. Un champ récepteur trop petit limite l'accès au contexte global, alors qu'un champ plus large favorise l'intégration spatiale au prix d'un coût calculatoire et paramétrique plus èlevé.

c) Hyperparamètres de régularisation.: Le dropout et la pénalisation L^2 doivent être analysés même lorsque leurs valeurs sont nulles dans le code. Un taux de dropout strictement positif force le réseau à ne pas dépendre d'un petit nombre de neurones particuliers, ce qui limite la co-adaptation et réduit le risque de sur-apprentissage. La pénalisation L^2, ou *weight decay*, décourage les poids trop grands et tend à lisser le modèle. Ces deux quantités sont donc bien des hyperparamètres de régularisation au sens du modèle, mais elles ne participent pas aux résultats observés dans le protocole courant puisque leurs valeurs sont nulles.

D. Synthèse

La Section V relève les hyperparamètres du projet un par un, avec leur nom exact dans le code, leur nature et leur influence attendue. Les hyperparamètres structurels imposés par la tâche doivent être distingués des hyperparamètres ajustables de conception, d'entraînement et de régularisation. Les tailles de sous-ensembles, les graines et le prétraitement conditionnent l'expérience, mais ils relèvent de conditions expérimentales externes plutôt que d'hyperparamètres du modèle lui-même.

References

[1] Keras, "Input object," Keras 3 API documentation, accessed Mar. 8, 2026. [Online]. Available: https://keras.io/api/layers/core_layers/input/

[2] Y. LeCun, K. Kavukcuoglu, and C. Farabet, "Convolutional Networks and Applications in Vision," presented at ISCAS 2010, accessed Mar. 8, 2026. [Online]. Available: https://cseweb.ucsd.edu/~gary/cs200/s12/lecun-iscas-10.pdf

[3] A. Krizhevsky, "CIFAR-10 and CIFAR-100 datasets," accessed Mar. 8, 2026. [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html

[4] Keras, "Conv2D layer," Keras 3 API documentation, accessed Mar. 8, 2026. [Online]. Available: https://keras.io/api/layers/convolution_layers/convolution2d/

[5] Keras, "Model training APIs," Keras 3 API documentation, accessed Mar. 8, 2026. [Online]. Available: https://keras.io/api/models/model_training_apis/

[6] Keras, "Input object," Keras 3 API documentation, accessed Mar. 8, 2026. [Online]. Available: https://keras.io/api/layers/core_layers/input/

[7] A. Manzanera, "Classification et apprentissage," support de cours, ENSTA Paris, 2026.

[8] A. Manzanera, "Chapitre 4 – Classification et apprentissage," polycopié de cours, ENSTA Paris, 2026.

TABLE IV

INVENTAIRE DÉTAILLÉ DES HYPERPARAMÈTRES STRUCTURELS, DE CONCEPTION ET DE RÉGULARISATION.

Nom dans le code	Nature précise	Valeur retenue	Influence attendue
`shape`	structurel imposé par la tâche	$32 \times 32 \times 3$	fixe la forme des données d'entrée et contraint la première couche du réseau
`units` (sortie)	structurel imposé par la tâche	10 sorties	impose la dimension de sortie du classifieur, donc le nombre de classes traitées
nombre de couches `Conv2D`	ajustable de conception	1 couche convolutionnelle	fixe la profondeur de l'extraction locale et contribue à la croissance du champ récepteur effectif
`filters`	ajustable de conception	8	modifie la largeur de la représentation convolutionnelle et le nombre de poids appris
`kernel_size`	ajustable de conception	$(3, 3)$	contrôle le voisinage local exploré et participe directement au champ récepteur
`padding`	ajustable de conception	`same`	préserve la résolution spatiale et conditionne la taille des tenseurs transmis aux couches suivantes
`pool_size`	ajustable de conception	$(2, 2)$	réduit la résolution spatiale, le coût calculatoire et agrandit le champ récepteur effectif des couches profondes
`Flatten`	ajustable de conception	`Flatten`	fixe la transition convolution/dense et donc la taille d'entrée de la tête de classification
`units` (Dense cachée)	ajustable de conception	64 neurones	contrôle la largeur de la tête dense et la capacité expressive de la partie pleinement connectée
`activation` (cachée)	ajustable de conception	ReLU	introduit la non-linéarité et limite en pratique l'atténuation du gradient dans les premières couches
`activation` (sortie)	ajustable de conception	softmax	rend la sortie compatible avec une interprétation probabiliste et avec l'entropie croisée
`rate`	régularisation	0.0	si le taux ètait strictement positif, il réduirait la co-adaptation des neurones ; ici son effet empirique est nul
`l2`	régularisation	$\lambda_{L^2} = 0.00$	si le coefficient ètait positif, il pénaliserait les poids trop grands ; ici son effet empirique est nul

TABLE V

INVENTAIRE DÉTAILLÉ DES HYPERPARAMÈTRES D'ENTRAÎNEMENT.

Nom dans le code	Nature précise	Valeur initiale	Valeurs explorées / retenues	Influence attendue
`loss`	entraînement	`categorical_crossentropy`	valeur conservée dans tout le protocole	définit la quantité dont le gradient pilote l'apprentissage ; ici elle est adaptée à une sortie softmax et à des labels one-hot
`optimizer`	entraînement	SGD	SGD, SGD+Momentum, Adam	modifie la dynamique de convergence, le lissage des gradients et la sensibilité au pas d'apprentissage
`learning_rate`	entraînement	0.01	0.01 pour SGD/SGD+Momentum ; 0.001 pour Adam	multiplie directement le gradient dans les mises à jour ; trop grand, il destabilise la descente ; trop faible, il ralentit la convergence
`momentum`	entraînement	0.0	0.0 puis 0.9	ajoute une inertie directionnelle qui lisse les fluctuations du gradient entre deux mini-batches
`beta_1`	entraînement	–	0.9 pour Adam	contrôle le lissage du premier moment dans Adam
`beta_2`	entraînement	–	0.999 pour Adam	contrôle le lissage du second moment dans Adam
`epsilon` d'Adam	entraînement	–	10^{-7} pour Adam	stabilise numériquement le dénominateur de l'update d'Adam ; il correspond au terme noté η dans le rapport
`batch_size`	entraînement	32	8 ; puis $\{8, 16, 32, 64, 128\}$; 32 retenu pour l'étude des optimiseurs	agit sur le bruit du gradient, le nombre d'updates par epoch et le compromis entre stabilité et coût calculatoire
`epochs`	entraînement	20	10 ; puis 8 retenu pour les comparaisons contrôlées	fixe le budget d'apprentissage en passages sur la base et conditionne le niveau de convergence atteint
`shuffle`	entraînement	valeur par défaut dans le notebook	`True` dans la pipeline reproductible	limite les effets d'ordre entre epochs et stabilise l'estimation du gradient stochastique